

Relatório — Modelagem e Implementação do Banco de Dados tipo LinkedIn

O presente relatório descreve o processo de modelagem e implementação de um banco de dados relacional desenvolvido em PostgreSQL, inspirado na estrutura de uma plataforma profissional similar ao LinkedIn. O objetivo foi criar uma base sólida e normalizada que suporte funcionalidades como criação de perfis, conexões entre usuários e organizações, publicações, candidaturas a vagas, troca de mensagens e registro de atividades, garantindo integridade referencial e desempenho otimizado para consultas complexas.

1. Principais Tabelas Criadas e Suas Funções

- **Usuario** – Armazena informações pessoais dos usuários, como nome, sobrenome, pronomes e data de nascimento. É a tabela central do sistema, representando cada perfil individual.
- **Organizacao** – Representa as organizações que possuem páginas dentro da plataforma. Contém dados como nome, setor, descrição e localidade.
- **Formacao** – Guarda os dados educacionais dos usuários, como instituição, diploma e período.
- **Experiencia** – Experiências que podem ser associadas a um usuario
- **Competencia** – Lista as habilidades (skills) que podem ser associadas aos usuários e também às vagas.
- **Perfil** - Armazena detalhes visuais e de apresentação do perfil, como título, foto e cargo atual.
- **Contato** - Armazena os contatos do usuário
- **Vaga** - Representa oportunidades de emprego cadastradas por empresas ou outros usuarios..
- **Conexao_seguir** - Relaciona usuários e organizações entre si, representando conexões (seguindo o modelo de rede social).
- **Publicacao** - Registra postagens feitas por usuários ou organizações.
- **Usuario_experiencia** - Contém o histórico profissional dos usuários, relacionando-os às empresas e funções desempenhadas.
- **Candidatura** - Relaciona usuários às vagas às quais se candidataram.
- **Localidade** - Endereço simples(pais, cidade) que pode ser associado a usuario e organizações
- **Formacao_usuario, Usuario_experiencia, Competencia_usuario** - tabelas associativas que tratam relações N:N entre usuários e formações, experiências e competências.
- **Atividade** – cataloga os tipos de ações possíveis dentro da rede (reação, comentário, etc.).
- **Atividade_publicacao** - registra interações (curtidas, comentários, compartilhamentos) em publicações.
- **Mensagem** – permite a comunicação privada entre usuários e/ou empresas.

2. Definição das Chaves Primárias e Estrangeiras

As tabelas possuem chaves primárias do tipo SERIAL para identificação única. Algumas tabelas tem chave composta. As chaves estrangeiras foram definidas para manter integridade referencial entre tabelas relacionadas.

- **PRIMARY KEY**: garante unicidade de registros.
- **FOREIGN KEY**: assegura integridade entre entidades (ex: usuário → perfil).
- **ON DELETE CASCADE**: exclui dependentes automaticamente. As relações dependentes (como perfil, contato, competencia_usuario) utilizam ON DELETE CASCADE, pois perdem sentido sem o registro pai.
- **ON DELETE SET NULL**: preserva dados históricos quando o vínculo é opcional.

3. Tipos de Dados Utilizados e Justificativas

Os tipos de dados foram definidos conforme eficiência e coerência com o tipo de informação armazenada:

- **VARCHAR(n)** - Nomes, e-mails e cargos - controle de tamanho e melhor desempenho.
- **SERIAL** - IDs primarios - Geração automática de identificadores únicos
- **TEXT** - Descrições longas - usado em publicações e mensagens.
- **DATE/TIMESTAMP** - Datas e horários - permite ordenação e análise temporal.
- **NUMERIC(12,2)** - Salários - garante precisão monetária.
- **BOOLEAN** - Campos lógicos - indica status de registros.
- **ENUM** - Categorias fixas (ex: tipo_localidade, tipo_emprego) - garante padronização.
- **INT / SMALLINT** - Relações e campos numéricos simples

4. Relacionamentos Muitos-para-Muitos

Os relacionamentos muitos-para-muitos foram tratados por meio de tabelas associativas:

- **usuario_experiencia** - conecta usuários e experiências profissionais;
- **formacao_usuario** - relaciona usuários e formações;
- **conexao_seguir** - modela múltiplos vínculos entre diferentes tipos de perfis;
- **competencia_usuario** - conecta usuários às competências, podendo também relacioná-las a formações, experiências ou organizações.

Isso garante flexibilidade e mantém o banco normalizado.

5. Uso de Constraints (Restrições)

Foram aplicadas diversas constraints para garantir integridade e regras de negócio:

- **PRIMARY KEY** - Garante unicidade dos registros.
- **FOREIGN KEY** - Mantém integridade referencial entre entidades.
- **UNIQUE** - Evita duplicação em campos críticos (ex: email).
- **CHECK** - Assegura condições lógicas, como publicações feitas por usuário OU organização.
- **DEFAULT** - Preenche automaticamente campos como data de criação.
- **NOT NULL** - Evita registros incompletos.

6. Índices e Otimização de Consultas

Foram criados diversos índices com o objetivo de otimizar as consultas mais frequentes da aplicação, garantindo eficiência nas buscas, listagens e ordenações.

Exemplos:

- **idx_usuario_email**: usado na autenticação e recuperação de perfis.
- **idx_publicacao_data**: garante ordenação eficiente no feed de publicações.
- **idx_vaga_titulo**: agiliza pesquisas de vagas por nome.
- **idx_competencia_nome**: melhora desempenho ao buscar usuários por habilidades.

Índices em colunas de chaves estrangeiras (usuario_id, organizacao_id) aceleram JOINS em consultas complexas. A criação dos índices foi planejada de acordo com a frequência de leitura e o tipo de operação predominante, equilibrando desempenho e custo de escrita (INSERT/UPDATE)